# Materiais e Métodos — Detecção de EPI com Raspberry Pi

Oct 21, 2026 · Monkey Tech

Um Raspberry Pi 3B processa o vídeo de uma webcam localmente, detecta capacete e colete com um modelo TFLite, aciona um buzzer no GPIO17 e registra cada violação confirmada em SQLite, Google Sheets, Google Drive e Looker Studio.

## Hardware

O custo de aquisição do protótipo foi de R$ 482,38; teclado, mouse, monitor, webcam e protoboard já pertenciam ao grupo.

| Item | Modelo / especificação | Função no projeto | Custo (R$) |
|---|---|---|---|
| Raspberry Pi 3B + fonte | Raspberry Pi 3 Model B, 1 GB de RAM, SoC BCM2837, Wi-Fi integrado; fonte oficial 5 V / 2,5 A micro-USB | Executa a inferência, o buzzer, o banco local e a sincronização | 379,91 |
| Cartão microSD | 32 GB, classe 10 / A1 | Sistema operacional, modelo, banco SQLite e fotos | 49,51 |
| Extensor de GPIO | Placa de extensão do header de 40 pinos | Leva os sinais do GPIO até a protoboard | 33,96 |
| Módulo buzzer | Buzzer ativo 5 V com circuito de acionamento embutido | Alerta sonoro: intermitente se falta 1 EPI, contínuo se faltam 2 | 19,00 |
| **Total adquirido** | | | **482,38** |

Itens que o grupo já possuía e que fazem parte do protótipo:

| Item | Modelo / especificação | Função no projeto |
|---|---|---|
| Webcam | Logitech BRIO, USB | Captura o vídeo em tempo real e as fotos do dataset |
| Protoboard e jumper wires | Protoboard padrão | Ligação do buzzer ao extensor de GPIO |

| Item | Modelo / especificação | Função no projeto |
|---|---|---|
| Monitor e cabo HDMI | Entrada HDMI, cabo padrão | Exibe a imagem com a detecção |
| Teclado e mouse USB | Padrão | Operação do Raspberry Pi |

## Software

O Raspberry Pi só executa a inferência e o registro; a anotação (Roboflow) e o treinamento (Google Colab) rodam no navegador de um notebook.

| Etapa | Ferramenta | Função no projeto |
|---|---|---|
| Sistema operacional | Raspberry Pi OS with desktop | Sistema do Pi; a versão com interface gráfica exibe o vídeo no monitor |
| Linguagem | Python 3 | Linguagem dos três scripts: `detect.py`, `registro.py` e `cloud_sync.py` |
| Imagem | OpenCV e NumPy | Leitura da webcam, redimensionamento do quadro, desenho das caixas e gravação dos snapshots |
| Inferência | ai-edge-litert (interpretador TFLite) | Executa o `best.tflite` no Pi, sem MediaPipe |
| Alerta | gpiozero | Controla o buzzer no GPIO17 (desligado, intermitente ou contínuo) |
| Banco local | SQLite (módulo `sqlite3` do Python) | Histórico offline das violações |
| Nuvem | gspread, google-api-python-client, google-auth e google-auth-oauthlib | Envio dos registros ao Google Sheets e das fotos ao Google Drive |
| Acesso ao Pi | SSH e SCP, a partir do Windows PowerShell | Administração remota e cópia do `best.tflite` |

| Etapa | Ferramenta | Função no projeto |
|---|---|---|
| Anotação | Roboflow | Organização do dataset, marcação das caixas e exportação em Pascal VOC |
| Treinamento | Google Colab Pro, `tflite-model-maker` e Miniconda (Python 3.9) | Ajuste fino do detector e exportação do `best.tflite` |
| Dashboard | Google Looker Studio | Gráficos e tabelas ligados à planilha, para acompanhamento remoto pelo gerente |

No Pi, o `detect.py` lê a webcam, roda o modelo e atualiza a máquina de estados. Quando uma violação é confirmada, o `registro.py` grava o snapshot e a linha no SQLite, e uma thread de fundo chama o `cloud_sync.py` a cada 15 segundos para enviar os registros pendentes. Sem internet, os registros ficam pendentes e o vídeo não trava.

O projeto previa o MediaPipe, mas ele foi descartado por dois motivos. Os binários do pacote travam no Pi 3B com o erro "Illegal instruction", porque exigem instruções de criptografia AES que o chip BCM2837 não possui. Além disso, o MediaPipe Model Maker exporta o modelo sem o pós-processamento (NMS) embutido, o que o torna incompatível com o interpretador TFLite puro. O treinamento passou a usar o `tflite-model-maker`, cujo modelo já sai com o NMS no próprio arquivo.

## Esquemático eletrônico

O único componente ligado ao GPIO é o módulo buzzer, com três fios (5 V, GND e sinal no GPIO17) que passam pelo extensor de GPIO e pela protoboard. O diagrama completo está no arquivo `esquematico_buzzer_raspberry_pi.pdf`, entregue à parte.

| Sinal | Pino físico do Pi | Nome / BCM | Terminal do buzzer |
|---|---|---|---|
| Alimentação | 2 | 5V | VCC |
| Terra | 6 | GND | GND |
| Controle | 11 | GPIO17 (BCM 17) | I/O |

Os pinos seguem a pinagem padrão do header de 40 pinos. O módulo tem lógica invertida: nível baixo no GPIO17 liga o som, por isso o código usa `Buzzer(17, active_high=False)` na gpiozero. O Pi é alimentado pela fonte micro-USB; a webcam usa a porta USB e o monitor, a saída HDMI.

# Dataset e modelo

O `best.tflite` foi gerado a partir de 493 imagens anotadas em 4 classes; o modelo atingiu AP50 de 100%, AP75 de 94,7% e AP geral (IoU de 0,50 a 0,95) de 78,7%, medidos sobre cerca de 500 imagens.

As fotos foram feitas com a webcam Logitech BRIO, com e sem capacete e com e sem colete, em condições variadas. A anotação foi feita no Roboflow (workspace `murilo-eidy`, projeto `my-first-project-silmy`, versão 1): cada imagem recebeu uma única caixa em volta da pessoa inteira, e a classe da caixa diz quais EPIs ela veste. Assim o modelo já classifica a situação da pessoa, e cada classe corresponde direto a um conjunto de EPIs faltantes na lógica do `detect.py`. O dataset foi exportado em Pascal VOC.

| Classe | Situação da pessoa | Violação registrada | Buzzer |
|---|---|---|---|
| `colete_capacete` | Usa capacete e colete | Nenhuma | Desligado |
| `so_colete` | Usa só o colete | `sem_capacete` | Intermitente |
| `so_capacete` | Usa só o capacete | `sem_colete` | Intermitente |
| `nenhum_EPI` | Não usa nenhum dos dois | `ambos` | Contínuo |

O treinamento rodou no Google Colab Pro com o `tflite-model-maker`, dentro de um ambiente Miniconda com Python 3.9 instalado manualmente, porque o Python padrão do Colab (3.12 ou 3.13) é incompatível com a biblioteca. O modelo base é o EfficientDet-Lite0, o menor da família EfficientDet-Lite.

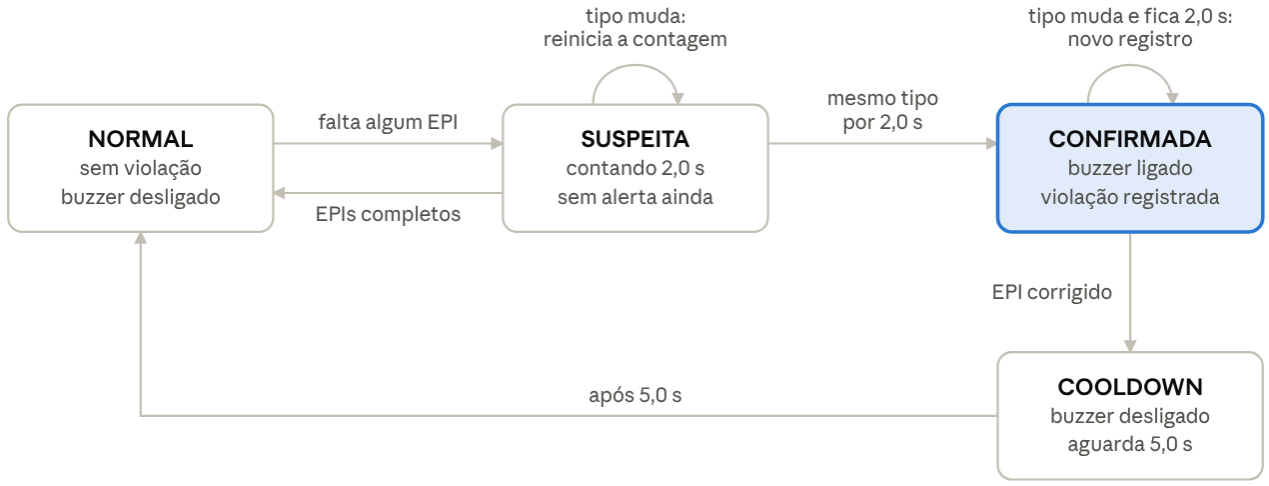
Ao fim do treinamento, `model.export()` gera o `best.tflite` com o NMS embutido no grafo, de modo que o modelo devolve quatro tensores: caixas, classes, pontuações e contagem de detecções. O arquivo é copiado para o Google Drive (`MyDrive/projeto_epi/`) logo após a exportação, como proteção contra quedas de sessão do Colab, e depois enviado ao Pi por SCP para `/home/epidetect/best.tflite`. Como a ordem dos tensores de saída não é confiável, o `detect.py` identifica cada um pelo formato e confere se as pontuações ficam entre 0 e 1.

Essas métricas vieram de cerca de 500 imagens, praticamente o dataset inteiro (493). Se esse conjunto incluir imagens usadas no treino, os valores medem o ajuste aos dados já vistos e não o desempenho em imagens novas, e o AP50 de 100% não garante o mesmo acerto em campo.

# Lógica de decisão

O modelo classifica a pessoa a cada quadro, mas o sistema só confirma uma violação depois de 2,0 s seguidos com o mesmo EPI faltando; assim, uma classificação que oscila por alguns quadros não dispara o alarme.

**Uma violação só vira alerta depois de 2,0 s ininterruptos**




máquina de estados · 4 estados

Em cada quadro, o `detect.py` descarta as detecções com confiança abaixo de 0,5 e usa a de maior confiança, já que há uma pessoa por quadro. A classe dessa detecção define quais EPIs faltam (tabela de classes da seção anterior). Ao confirmar, o buzzer pisca se falta 1 EPI e toca continuamente se faltam 2; o snapshot e o registro vão para o SQLite. Em CONFIRMADA, se o tipo de violação muda e se mantém por 2,0 s, o sistema registra uma nova violação e atualiza o buzzer.

| Parâmetro | Valor | Efeito |
|---|---|---|
| `CONFIDENCE_THRESHOLD` | 0,5 | Detecções abaixo dessa confiança são descartadas |
| `TEMPO_CONFIRMACAO` | 2,0 s | Tempo contínuo com o mesmo EPI faltando para confirmar a violação |
| `TEMPO_COOLDOWN` | 5,0 s | Espera após a correção antes de voltar ao estado NORMAL |
| `BIP_ON` / `BIP_OFF` | 0,4 s / 0,4 s | Duração do bip e da pausa no buzzer intermitente |
| `INTERVALO_SYNC` | 15 s | Intervalo entre tentativas de envio dos pendentes à nuvem |

## Banco de dados e nuvem

Cada violação confirmada é gravada primeiro no SQLite local e só depois enviada ao Google Sheets e ao Google Drive; assim, uma queda de internet não interrompe a detecção nem faz o sistema perder registros.

O banco `violacoes.db` tem uma tabela, `violacoes`:

| Campo | Tipo | Conteúdo |
|---|---|---|
| `id` | INTEGER, chave primária | Identificador sequencial do registro |
| `timestamp` | TEXT | Data e hora da violação (AAAA-MM-DD HH:MM:SS) |
| `tipo` | TEXT | `sem_capacete`, `sem_colete` ou `ambos` |
| `caminho_foto` | TEXT | Caminho do snapshot salvo em `fotos_violacoes/` |
| `enviado_nuvem` | INTEGER | 0 = pendente de envio, 1 = já enviado |

A função `sincronizar_pendentes()`, do `cloud_sync.py`, busca os registros com `enviado_nuvem = 0` em ordem de ID. Para cada um, sobe a foto para uma pasta do Google Drive, acrescenta uma linha na planilha "Registros EPI" (data e hora, tipo da violação e link da foto no Drive) e marca o registro como enviado. Se o envio falha, a rotina para e retoma na rodada seguinte, 15 segundos depois.

O Google Sheets é acessado por uma conta de serviço. O Google Drive é acessado por OAuth com a conta do próprio usuário, porque contas de serviço não têm cota de armazenamento em contas pessoais do Google. As credenciais ficam em arquivos separados no Pi, fora do código. No teste com o Wi-Fi desligado, as violações ficaram pendentes no SQLite sem travar o vídeo.

## Dashboard

O dashboard "Monitoramento de EPI", feito no Google Looker Studio e ligado à planilha "Registros EPI", mostra ao gerente as violações por tipo e por horário do dia, com as fotos de evidência.

| Componente | O que mostra |
|---|---|
| Gráfico de pizza | Violações por tipo (sem capacete, sem colete e ambos) |

| Componente | O que mostra |
|---|---|
| Total de violações | Contagem geral de registros |
| Gráfico de colunas | Violações por hora do dia |
| Tabela de evidências | Registros com o link da foto no Google Drive |
| Filtros | Tipo de violação e período |

Os dados vêm da planilha, que é atualizada automaticamente pelo `cloud_sync.py`; por isso o dashboard acompanha as violações sem que o gerente precise acessar o Raspberry Pi.